CRUD Car Dealereship OOP

Contents

1 CRUD Car Dealership in OOP PHP

Rewriting the Car Dealership PHP CRUD application in an Object-Oriented Programming (OOP) style involves creating classes to represent entities like cars, clients, and purchases and encapsulating the CRUD operations within these classes. Below is a simplified example to demonstrate this approach.

1.1 Define the `Car` Class

This class represents a car and includes methods for CRUD operations related to cars.

```php
<?=
class Car {
    private $pdo;

    public function __construct($pdo) {
        $this->pdo = $pdo;
    }
```

```
    public function create($model, $price, $brand) {
        // Insert a new car into the database
    }

    public function read($id) {
        // Retrieve a car from the database by ID
    }

    public function update($id, $model, $price, $brand) {
        // Update car details in the database
    }

    public function delete($id) {
        // Delete a car from the database
    }
}
```

1.2 Implement CRUD Methods

Each method in the `Car` class should perform the corresponding database operation. For example, the `create` method could be implemented as follows:

```
<?=
public function create($model, $price, $brand) {
    $stmt = $this->pdo->prepare("INSERT INTO cars (model, price, brand) VALUES (?
    $stmt->execute([$model, $price, $brand]);
}
```

Similarly, you would implement the `read`, `update`, and `delete` methods.

1.3 Create Other Classes

Follow a similar pattern to create classes for other entities like `Client` and `Purchase`. Each class should have its own CRUD methods.

1.4 Use the Classes in Your Application

To use these classes in your application:

1. **Establish a Database Connection:** Create a PDO connection to your database and pass it to the constructor of your entity classes.

   ```
   <?=
   $pdo = new PDO("mysql:host=localhost;dbname=car_dealership", "username", "pas
   ```

2. **Perform Operations:** Create instances of your classes and call their methods to perform operations.

   ```
   <?=
    $car = new Car($pdo);
   ```

```php
$car->create("Tesla Model S", 79900, "Tesla");

$carToUpdate = 1;
$car->update($carToUpdate, "Tesla Model S Plaid", 129990, "Tesla");

$carToDelete = 2;
$car->delete($carToDelete);
```

1.5 Conclusion

In this OOP approach, each class corresponds to an entity in your application, and the CRUD operations are methods within these classes. This structure leads to more organized and maintainable code, especially for larger applications. Each class is responsible for the operations related to a single aspect of the application, adhering to the Single Responsibility Principle, a key tenet of OOP.

2 Continuation

To complete the Car Dealership CRUD application in an OOP style, we'll create `Dealer` and `Purchase` classes. These classes will handle operations related to car dealers and purchases, respectively. We'll also outline the necessary HTML modifications for integrating these classes into the application.

2.1 `Dealer` Class

This class manages dealer-related operations.

```php
class Dealer {
    private $pdo;

    public function __construct($pdo) {
        $this->pdo = $pdo;
    }

    public function sellCar($carId, $clientId, $dealerId) {
        // Assuming the method to register a car sale
        // This could include reducing the inventory count for the car
        // and adding an entry to the purchase history

        // Example of adding an entry to a purchase table
        $stmt = $this->pdo->prepare("INSERT INTO purchases (car_id, client_id, de
        $stmt->execute([$carId, $clientId, $dealerId]);
    }

    // Additional methods for dealer operations can be added here
}
```

2.2 `Purchase` Class

This class encapsulates operations related to purchases.

```
class Purchase {
    private $pdo;

    public function __construct($pdo) {
        $this->pdo = $pdo;
    }

    public function getPurchaseDetails($purchaseId) {
        $stmt = $this->pdo->prepare("SELECT * FROM purchases WHERE id = ?");
        $stmt->execute([$purchaseId]);
        return $stmt->fetch();
    }

    // Additional methods for handling purchase operations
}
```

2.3 HTML Modifications

To integrate these new classes into your web application, you'll need to modify the HTML to allow for dealer operations like selling a car. Here's an example of how you might structure a simple form for a dealer to record a car sale:

2.3.1 HTML Form for Selling a Car (`sell_car.html`)

```
<!DOCTYPE html>
<html>
<head>
    <title>Sell a Car</title>
</head>
<body>
    <h1>Sell a Car</h1>
    <form action="sell_car.php" method="post">
        Car ID: <input type="number" name="car_id"><br>
        Client ID: <input type="number" name="client_id"><br>
        Dealer ID: <input type="number" name="dealer_id"><br>
        <input type="submit" value="Sell Car">
    </form>
</body>
</html>
```

2.3.2 PHP Script for Handling the Form (`sell_car.php`)

```
<?php
require_once 'db_connect.php'; // Database connection
```

```php
require_once 'Dealer.php';

$pdo = new PDO("mysql:host=localhost;dbname=car_dealership", "username", "passwor

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $dealer = new Dealer($pdo);
    $dealer->sellCar($_POST['car_id'], $_POST['client_id'], $_POST['dealer_id']);

    echo "Car sold successfully.";
}
?>
```

2.4 Conclusion

In the OOP version of your Car Dealership application, `Dealer` and `Purchase` classes encapsulate all the logic related to their respective domains. This makes your code more organized, modular, and easier to maintain.

You'll need to adjust your HTML forms to work with these classes. The example above shows how you can create a form for a dealer to sell a car, which is then processed by a PHP script using the `Dealer` class.

Remember, these are basic examples to demonstrate the concept. In a real-world application, you'd likely have more comprehensive error checking, input validation, and possibly more complex business logic incorporated into your class methods.